

GM TECH SOLUTIONS

Escola High Tech - Frontend

Documentacao tecnica do projeto front, separada do backend.

Escola High Tech - Frontend

Documentacao tecnica do frontend da aplicacao Escola High Tech. Este README cobre somente o projeto do front, separado do backend/API.

Visao geral

O projeto e uma SPA em Nuxt 3 para gerenciamento escolar. A aplicacao consome uma API REST externa e oferece telas de autenticacao, painel inicial, alteracao de senha e CRUD de:

- Alunos
- Professores
- Usuarios
- Diretoria

O layout atual usa uma identidade visual com fundo claro, cards brancos, destaque laranja para marca/secao, botoes em verde/teal e icones Lucide.

Tecnologias usadas

TECNOLOGIA	USO
Nuxt 3.21.6	Framework Vue, roteamento por arquivos, plugins e build
Vue 3	Camada de componentes e Composition API
TypeScript strict	Tipagem de stores, API e componentes
Pinia	Estado global de autenticacao e stores de entidades
Tailwind CSS	Estilizacao utilitaria das telas
ofetch / <code>\$fetch</code>	Cliente HTTP usado pelo wrapper <code>\$api</code>
@lucide/vue	Icones de interface
Vitest	Testes unitarios
happy-dom	Ambiente DOM para testes
@nuxt/test-utils	Configuracao de testes Nuxt
Nginx	Servir o build estatico no Docker

Estrutura principal

```
ESCOLA_FRONT/
|-- assets/css/main.css           # Tailwind e estilos globais legados
|-- components/                  # Componentes reutilizaveis
|-- layouts/                     # Layouts default e auth
|-- middleware/auth.global.ts    # Guarda global de autenticacao/autorizacao
|-- pages/                       # Rotas Nuxt
|-- plugins/api.ts              # Injeta o cliente HTTP $api
|-- stores/                      # Stores Pinia
|-- tests/                      # Testes unitarios
|-- types/api.ts                # Contratos TypeScript da API
|-- utils/                      # Utilitarios de API e senha
-- Dockerfile                   # Build estatico + Nginx
-- nuxt.config.ts               # Configuracao Nuxt
```

Configuracao

Variaveis suportadas:

VARIAVEL	PADRAO LOCAL	DESCRICAO
NUXT_PUBLIC_API_BASE	http://localhost:5001/api	URL base da API consumida pelo front
NUXT_APP_BASE_URL	/	Base URL da aplicacao Nuxt

No Docker Compose do projeto, o front e publicado em `http://localhost:8080` e a API e apontada para `http://localhost:5000/api`.

Como executar

Instalar dependencias:

```
npm install
```

Rodar em desenvolvimento:

```
npm run dev
```

Gerar build de producao:

```
npm run build
```

Gerar build estatico:

```
npm run generate
```

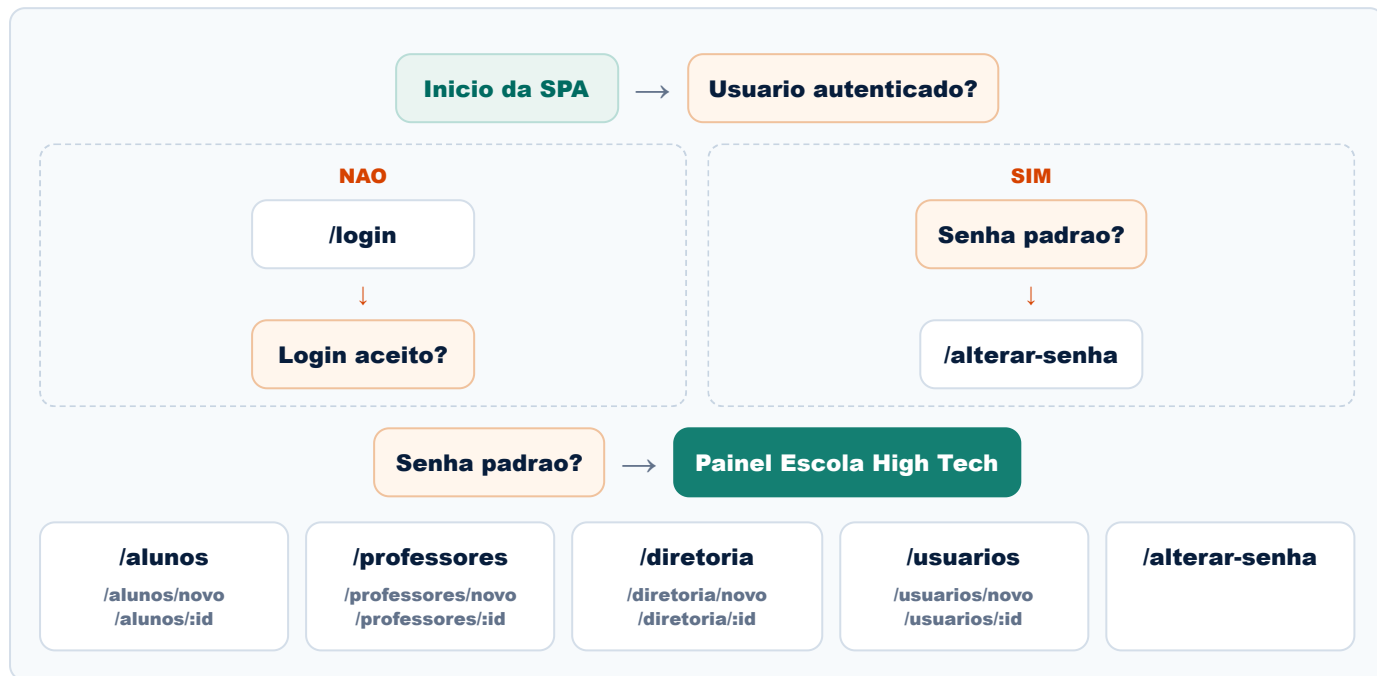
Rodar testes:

```
npm run test
```

Com Docker Compose, a partir da pasta `docker` do repositório:

```
docker compose up -d --build front
```

Fluxo de navegacao



Telas

ROTA	FINALIDADE
/login	Login e troca obrigatoria da senha padrao apos autenticacao
/	Painel com atalhos para os modulos
/alterar-senha	Alteracao manual de senha
/alunos	CRUD inline de alunos, busca, paginacao e acoes
/alunos/novo	Cadastro dedicado de aluno
/alunos/:id	Visualizacao, edicao e exclusao de aluno
/professores	CRUD inline de professores, busca, paginacao e acoes
/professores/novo	Cadastro dedicado de professor
/professores/:id	Visualizacao, edicao e exclusao de professor
/diretoria	CRUD inline da diretoria, busca, paginacao e acoes
/diretoria/novo	Cadastro dedicado de integrante da diretoria
/diretoria/:id	Visualizacao, edicao e exclusao de integrante
/usuarios	Consulta e CRUD de usuarios conforme perfil
/usuarios/novo	Cadastro dedicado de usuario
/usuarios/:id	Visualizacao, edicao e exclusao de usuario conforme perfil

Perfis de acesso

A sessao autenticada guarda o usuario logado, token JWT, data de expiracao e flag de senha padrao no `localStorage`, usando a chave `form-escola-auth`.

Os perfis reconhecidos no front sao:

- Administrador**: acesso completo no front; pode cadastrar, editar e excluir registros.
- Contribuinte**: pode acessar modulos operacionais e cadastrar/editar alunos, professores e diretoria. Para usuarios, a tela permite consulta; criacao, edicao e exclusao ficam restritas ao administrador.
- Outros perfis autenticados: podem acessar rotas gerais sem `roles`, como painel e alteracao de senha. Rotas com `roles` redirecionam para o painel quando o perfil nao e permitido.

Matriz resumida:

AREA	ADMINISTRADOR	CONTRIBUINTE	OUTROS AUTENTICADOS
Login	Sim	Sim	Sim
Painel	Sim	Sim	Sim
Alterar senha	Sim	Sim	Sim
Alunos	Criar, visualizar, editar, excluir	Criar, visualizar, editar	Restrito por rota/listagem
Professores	Criar, visualizar, editar, excluir	Criar, visualizar, editar	Restrito por rota/listagem
Diretoria	Criar, visualizar, editar, excluir	Criar, visualizar, editar	Restrito por rota/listagem
Usuarios	Criar, visualizar, editar, excluir	Visualizar	Restrito

Observacao importante: o front controla a experiencia e evita acoes indevidas na UI, mas a API deve continuar sendo a fonte final de autorizacao.

Autenticacao e autorizacao

Arquivos principais:

- `stores/auth.ts` : controla login, logout, persistencia da sessao, perfil, `isAdmin` e `canWrite` .
- `middleware/auth.global.ts` : guarda global de rotas.
- `plugins/api.ts` : injeta `$api` com token JWT e tratamento de `401` .
- `utils/api-client.ts` : monta headers, normaliza erros e executa chamadas HTTP.

Regras do middleware:

- Rotas publicas, como `/login` , nao exigem sessao.
- Usuario autenticado que acessa `/login` e redirecionado para o painel ou para `/alterar-senha` .
- Rotas protegidas sem token redirecionam para `/login` .
- Usuario com senha padrao e forçado a acessar `/alterar-senha` .
- Rotas com `meta.roles` validam `usuario.descricaoPerfil` .

Integracao com a API

O cliente `$api` usa `NEXT_PUBLIC_API_BASE` como baseUrl e envia `Authorization: Bearer <token>` quando existe token.

Endpoints consumidos pelo front:

MODULO	ENDPOINTS
Auth	<code>POST /auth/login</code> , <code>GET /auth/me</code> , <code>POST /auth/alterar-senha</code>
Usuarios	<code>GET /usuarios</code> , <code>GET /usuarios/:id</code> , <code>POST /usuarios</code> , <code>PUT /usuarios/:id</code> , <code>DELETE /usuarios/:id</code> , <code>GET /usuarios/perfis</code>
Alunos	<code>GET /aluno</code> , <code>GET /aluno/:id</code> , <code>POST /aluno</code> , <code>PUT /aluno/:id</code> , <code>DELETE /aluno/:id</code>
Professores	<code>GET /professor</code> , <code>GET /professor/:id</code> , <code>POST /professor</code> , <code>PUT /professor/:id</code> , <code>DELETE /professor/:id</code>
Diretoria	<code>GET /diretoria</code> , <code>GET /diretoria/:id</code> , <code>POST /diretoria</code> , <code>PUT /diretoria/:id</code> , <code>DELETE /diretoria/:id</code>

Os contratos esperados das respostas ficam em `types/api.ts` .

Validacoes

Nao ha uma biblioteca externa dedicada a validacao de formularios, como Zod, Yup ou VeeValidate. O projeto usa:

- Validacao nativa do HTML (`required` , `type` , `maxlength`).
- Tipagem TypeScript para contratos de payload e resposta.
- Utilitario proprio `utils/password-strength.ts` para avaliar forza da senha.
- `normalizeApiError` em `utils/api-client.ts` para exibir mensagens de erro da API, incluindo erros de validacao retornados no formato `{ errors: ... }`.

Regras de forza de senha:

- Minimo de 8 caracteres.
- Pelo menos uma letra maiuscula.
- Pelo menos uma letra minuscula.
- Pelo menos um numero.
- Pelo menos um caractere especial.

O componente `components/PasswordStrengthMeter.vue` mostra a classificacao: `Nao informada` , `Fraca` , `Media` ou `Forte` .

Testes unitarios

Configuracao:

- `vitest.config.ts` usa `defineVitestConfig` de `@nuxt/test-utils/config` .
- Ambiente de teste: `happy-dom` .
- Setup global: `tests/setup.ts` , limpando `localStorage` e mocks antes de cada teste.

Cobertura atual:

ARQUIVO	O QUE VALIDA
<code>tests/stores/auth.spec.ts</code>	Login, persistencia de JWT, logout e alteracao de senha
<code>tests/utils/api-client.spec.ts</code>	Header Authorization, preservacao de headers, callback de 401 e normalizacao de erros
<code>tests/utils/password-strength.spec.ts</code>	Classificacao de senha vazia, fraca e forte

Executar:

```
npm run test
```

Build e deploy do front

O projeto esta configurado como SPA com `ssr: false` .

Fluxo do Dockerfile:

- Usa `node:22` para instalar dependencias.
- Executa `npm run generate` .
- Copia `.output/public` para uma imagem `nginx:alpine` .
- O Nginx serve a SPA e redireciona rotas internas para `index.html` .

Comando local do container via Compose:

```
docker compose -f ../docker/docker-compose.yml up -d --build front
```

Padrao visual

O layout principal fica em `layouts/default.vue` :

- Marca `GM Tech Solutions` no topo.
- Titulo dinamico conforme a rota.
- Nome do usuario autenticado.
- Botoes `Alterar senha` e `Sair` com icones.

O layout de autenticacao fica em `layouts/auth.vue` e centraliza a tela de login.

As telas principais de CRUD seguem o padrao:

- Card de cadastro/edicao a esquerda.
- Card de listagem a direita.
- Campo de busca.
- Tabela com acoes por icone.
- Paginacao visual.
- Botao de atualizar lista.

Scripts disponiveis

SCRIPT	ACAO
<code>npm run dev</code>	Sobe o servidor de desenvolvimento Nuxt
<code>npm run build</code>	Build de producao Nuxt
<code>npm run generate</code>	Gera build estatico para hospedagem
<code>npm run preview</code>	Preview do build
<code>npm run test</code>	Executa testes unitarios
<code>npm run test:watch</code>	Executa Vitest em modo watch
<code>npm run postinstall</code>	Gera tipos Nuxt apos instalar dependencias

Observacoes de manutencao

- Este README documenta somente o front. Regras de banco de dados, migrations e endpoints internos pertencem ao backend.
- O controle de acesso no front melhora a experiencia, mas nao substitui validacao/autorizacao no backend.
- Ao adicionar nova rota protegida, definir `definePageMeta({ roles: [...] })` quando a tela tiver restricao por perfil.
- Ao adicionar novos endpoints, atualizar `types/api.ts` , stores/paginas correspondentes e testes quando houver logica compartilhada.